

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №4
По дисциплине "Вычислительная математика"
Вариант №2

Выполнил студент группы Р3209
Евграфов Артём Андреевич
Проверил:
Рыбаков Степан Дмитриевич

Санкт-Петербург 2026

Содержание

1. Вычислительная реализация задачи	2
1.1. Таблица исходных данных	2
1.2. Линейная аппроксимация: $\varphi(x) = ax + b$	2
1.3. Квадратичная аппроксимация: $\varphi(x) = a_2x^2 + a_1x + a_0$	2
1.4. Вывод по вычислительной части	3
2. Листинг программы	3
3. Выводы	10

1. Вычислительная реализация задачи

Задана функция $y = \frac{3x}{x^4+1}$. Интервал исследования $x \in [0, 2]$ с шагом $h = 0,2$. Количество точек $n = 11$.

1.1. Таблица исходных данных

Вычислим значения функции в заданных точках (округление до трех знаков):

i	1	2	3	4	5	6	7	8	9	10	11
x_i	0,0	0,2	0,4	0,6	0,8	1,0	1,2	1,4	1,6	1,8	2,0
y_i	0,000	0,599	1,170	1,593	1,703	1,500	1,171	0,867	0,635	0,470	0,353

1.2. Линейная аппроксимация: $\varphi(x) = ax + b$

СЛАУ для линейной функции:

$$\begin{cases} a \cdot SXX + b \cdot SX = SXY \\ a \cdot SX + b \cdot n = SY \end{cases}$$

Вычислим необходимые суммы: $SX = \sum x_i = 11,0$; $SXX = \sum x_i^2 = 15,400$; $SY = \sum y_i = 10,061$; $SXY = \sum x_i y_i = 9,593$; $n = 11$.

Подставляем в систему:

$$\begin{cases} 15,400a + 11,000b = 9,593 \\ 11,000a + 11,000b = 10,061 \end{cases}$$

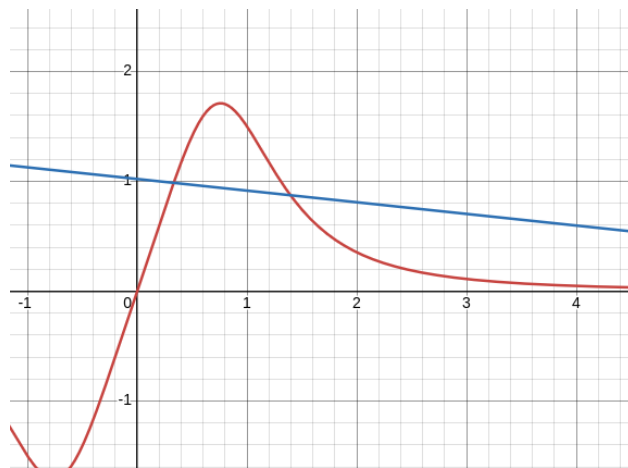
Решая систему, находим $a = \frac{\Delta_1}{\Delta} \approx -0,106$ и $b = \frac{\Delta_2}{\Delta} \approx 1,021$.

Итоговая линейная функция: $\varphi_1(x) = -0,106x + 1,021$.

Мера отклонения $S_1 = \sum (\varphi_1(x_i) - y_i)^2 \approx 3,035$.

Среднеквадратичное отклонение: $\delta_1 = \sqrt{\frac{3,035}{11}} \approx 0,525$.

Коэффициент корреляции Пирсона: $r = -0,157$ (связь слабая).



1.3. Квадратичная аппроксимация: $\varphi(x) = a_2x^2 + a_1x + a_0$

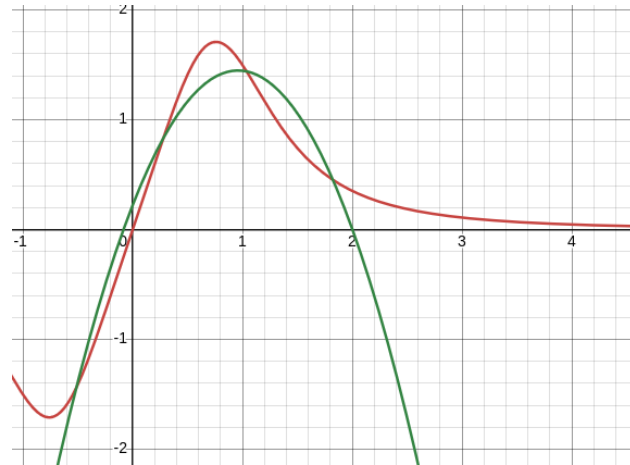
СЛАУ для квадратичной функции:

$$\begin{cases} a_0n + a_1 \sum x_i + a_2 \sum x_i^2 = \sum y_i \\ a_0 \sum x_i + a_1 \sum x_i^2 + a_2 \sum x_i^3 = \sum x_i y_i \\ a_0 \sum x_i^2 + a_1 \sum x_i^3 + a_2 \sum x_i^4 = \sum x_i^2 y_i \end{cases}$$

Вычислив суммы (дополнительно $\sum x_i^3 = 24,200$, $\sum x_i^4 = 40,533$, $\sum x_i^2 y_i = 11,320$) и решив систему методом Гаусса, получаем: $a_2 \approx -1,332$, $a_1 \approx 2,558$, $a_0 \approx 0,221$.
Итоговая квадратичная функция: $\varphi_2(x) = -1,332x^2 + 2,558x + 0,221$.

Мера отклонения $S_2 \approx 0,597$.

Среднеквадратичное отклонение: $\delta_2 = \sqrt{\frac{0,597}{11}} \approx 0,233$.



1.4. Вывод по вычислительной части

Сравнивая среднеквадратичные отклонения, видим, что $\delta_2(0,233) < \delta_1(0,525)$. Следовательно, квадратичная аппроксимация описывает заданную функцию точнее, чем линейная. Поскольку в исходных данных присутствуют $x = 0$ и $y = 0$, логарифмическая, степенная и экспоненциальная модели неприменимы из-за ограничений области допустимых значений.

2. Листинг программы

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <iomanip>
5  #include <string>
6  #include <limits>
7  #include <fstream>
8  #include <algorithm>
9  #include "matplotlibcpp.h"
10
11 namespace plt = matplotlibcpp;
12 using namespace std;
13
14 struct Point {
15     double x;
16     double y;
17 };
18
19 struct ApproxResult {
20     string name;
21     vector<double> coeffs;
22     double S;           // мера отклонения
23     double delta;       // ско
24     double R2;          // достоверность аппроксимации
25     bool valid = true;
26     string equation;

```

```

27 };
28
29
30 template <typename T>
31 T get_input(const string& prompt) {
32     T value;
33     while (true) {
34         cout << prompt;
35         if (cin >> value) {
36             if (cin.peek() == '\n' || cin.peek() == ' ') {
37                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
38                 return value;
39             }
40         }
41         cin.clear();
42         cin.ignore(numeric_limits<streamsize>::max(), '\n');
43         cout << "введено некорректное значение\n";
44     }
45 }
46
47 vector<double> solve_gauss(vector<vector<double>> matrix) {
48     int n = matrix.size();
49     for (int i = 0; i < n; ++i) {
50         int max_row = i;
51         for (int k = i + 1; k < n; k++) {
52             if (abs(matrix[k][i]) > abs(matrix[max_row][i])) max_row = k;
53         }
54         swap(matrix[i], matrix[max_row]);
55
56         if (abs(matrix[i][i]) < 1e-12) return {};
57
58         for (int k = i + 1; k < n; ++k) {
59             double c = -matrix[k][i] / matrix[i][i];
60             for (int j = i; j <= n; ++j) {
61                 if (i == j) matrix[k][j] = 0;
62                 else matrix[k][j] += c * matrix[i][j];
63             }
64         }
65     }
66
67     vector<double> x(n);
68     for (int i = n - 1; i >= 0; i--) {
69         x[i] = matrix[i][n] / matrix[i][i];
70         for (int k = i - 1; k >= 0; k--) {
71             matrix[k][n] -= matrix[k][i] * x[i];
72         }
73     }
74     return x;
75 }
76
77 double calc_R2(const vector<Point>& pts, const vector<double>& phi_vals) {
78     double sum_num = 0, sum_phi_sq = 0, sum_phi = 0;
79     int n = pts.size();
80     for (int i = 0; i < n; i++) {
81         sum_num += pow(pts[i].y - phi_vals[i], 2);
82         sum_phi_sq += pow(phi_vals[i], 2);
83         sum_phi += phi_vals[i];
84     }
85     double den = sum_phi_sq - (1.0 / n) * pow(sum_phi, 2);
86     if (abs(den) < 1e-12) return 0.0;
87     return 1.0 - (sum_num / den);

```

```

88 }
89
90 double pearson_correlation(const vector<Point>& pts) {
91     double sumX = 0, sumY = 0;
92     for(const auto& p : pts) { sumX += p.x; sumY += p.y; }
93     double meanX = sumX / pts.size();
94     double meanY = sumY / pts.size();
95
96     double num = 0, denX = 0, denY = 0;
97     for(const auto& p : pts) {
98         num += (p.x - meanX) * (p.y - meanY);
99         denX += pow(p.x - meanX, 2);
100        denY += pow(p.y - meanY, 2);
101    }
102    if (denX == 0 || denY == 0) return 0.0;
103    return num / sqrt(denX * denY);
104 }
105
106
107 ApproxResult calc_linear(const vector<Point>& pts) {
108     int n = pts.size();
109     double SX = 0, SY = 0, SXX = 0, SXY = 0;
110     for (const auto& p : pts) {
111         SX += p.x; SY += p.y;
112         SXX += p.x * p.x; SXY += p.x * p.y;
113     }
114     vector<vector<double>> matrix = {{SXX, SX, SXY}, {SX, (double)n, SY}};
115     vector<double> coeffs = solve_gauss(matrix);
116     if (coeffs.empty()) return {"Линейная", {}, 0, 0, 0, false, ""};
117
118     double S = 0;
119     vector<double> phi_vals(n);
120     for (int i = 0; i < n; i++) {
121         phi_vals[i] = coeffs[0] * pts[i].x + coeffs[1];
122         S += pow(phi_vals[i] - pts[i].y, 2);
123     }
124     return {"Линейная", coeffs, S, sqrt(S / n), calc_R2(pts, phi_vals), true, "y = " +
125         ↪ to_string(coeffs[0]) + "x + " + to_string(coeffs[1])};
126 }
127
128 ApproxResult calc_quad(const vector<Point>& pts) {
129     int n = pts.size();
130     double SX = 0, SX2 = 0, SX3 = 0, SX4 = 0, SY = 0, SXY = 0, SX2Y = 0;
131     for (const auto& p : pts) {
132         SX += p.x; SX2 += pow(p.x, 2); SX3 += pow(p.x, 3); SX4 += pow(p.x, 4);
133         SY += p.y; SXY += p.x * p.y; SX2Y += pow(p.x, 2) * p.y;
134     }
135     vector<vector<double>> matrix = {
136         {SX4, SX3, SX2, SX2Y}, {SX3, SX2, SX, SXY}, {SX2, SX, (double)n, SY}
137     };
138     vector<double> coeffs = solve_gauss(matrix);
139     if (coeffs.empty()) return {"Квадратичная", {}, 0, 0, 0, false, ""};
140
141     double S = 0;
142     vector<double> phi_vals(n);
143     for (int i = 0; i < n; i++) {
144         phi_vals[i] = coeffs[0] * pow(pts[i].x, 2) + coeffs[1] * pts[i].x + coeffs[2];
145         S += pow(phi_vals[i] - pts[i].y, 2);
146     }

```

```

146     return {"Квадратичная", coeffs, S, sqrt(S / n), calc_R2(pts, phi_vals), true, "y = "
    ↪ + to_string(coeffs[0]) + "x^2 + " + to_string(coeffs[1]) + "x + " +
    ↪ to_string(coeffs[2])});
147 }
148
149 ApproxResult calc_cubic(const vector<Point>& pts) {
150     int n = pts.size();
151     vector<double> sx(7, 0), sxy(4, 0);
152     for (const auto& p : pts) {
153         for (int i = 0; i <= 6; i++) sx[i] += pow(p.x, i);
154         for (int i = 0; i <= 3; i++) sxy[i] += pow(p.x, i) * p.y;
155     }
156     vector<vector<double>> matrix = {
157         {sx[6], sx[5], sx[4], sx[3], sxy[3]}, {sx[5], sx[4], sx[3], sx[2], sxy[2]},
158         {sx[4], sx[3], sx[2], sx[1], sxy[1]}, {sx[3], sx[2], sx[1], sx[0], sxy[0]}
159     };
160     vector<double> coeffs = solve_gauss(matrix);
161     if (coeffs.empty()) return {"Кубическая", {}, 0, 0, 0, false, ""};
162
163     double S = 0;
164     vector<double> phi_vals(n);
165     for (int i = 0; i < n; i++) {
166         phi_vals[i] = coeffs[0]*pow(pts[i].x,3) + coeffs[1]*pow(pts[i].x,2) +
    ↪ coeffs[2]*pts[i].x + coeffs[3];
167         S += pow(phi_vals[i] - pts[i].y, 2);
168     }
169     return {"Кубическая", coeffs, S, sqrt(S / n), calc_R2(pts, phi_vals), true, "y = "
    ↪ a3*x^3 + a2*x^2 + a1*x + a0"};
170 }
171
172 ApproxResult calc_exp(const vector<Point>& pts) {
173     vector<Point> lin_pts;
174     for (const auto& p : pts) {
175         if (p.y <= 0) return {"Экспоненциальная", {}, 0, 0, 0, false, "y <= 0"};
176         lin_pts.push_back({p.x, log(p.y)});
177     }
178     ApproxResult lin = calc_linear(lin_pts);
179     if (!lin.valid) return {"Экспоненциальная", {}, 0, 0, 0, false, ""};
180     double a = exp(lin.coeffs[1]), b = lin.coeffs[0], S = 0;
181     vector<double> phi_vals(pts.size());
182     for (size_t i = 0; i < pts.size(); i++) {
183         phi_vals[i] = a * exp(b * pts[i].x);
184         S += pow(phi_vals[i] - pts[i].y, 2);
185     }
186     return {"Экспоненциальная", {a, b}, S, sqrt(S / pts.size()), calc_R2(pts, phi_vals),
    ↪ true, "y = " + to_string(a) + " * e^(" + to_string(b) + "x)"};
187 }
188
189 ApproxResult calc_log(const vector<Point>& pts) {
190     vector<Point> lin_pts;
191     for (const auto& p : pts) {
192         if (p.x <= 0) return {"Логарифмическая", {}, 0, 0, 0, false, "x <= 0"};
193         lin_pts.push_back({log(p.x), p.y});
194     }
195     ApproxResult lin = calc_linear(lin_pts);
196     if (!lin.valid) return {"Логарифмическая", {}, 0, 0, 0, false, ""};
197     double a = lin.coeffs[0], b = lin.coeffs[1], S = 0;
198     vector<double> phi_vals(pts.size());
199     for (size_t i = 0; i < pts.size(); i++) {
200         phi_vals[i] = a * log(pts[i].x) + b;
201         S += pow(phi_vals[i] - pts[i].y, 2);

```

```

202     }
203     return {"Логарифмическая", {a, b}, S, sqrt(S / pts.size()), calc_R2(pts, phi_vals),
        ↪ true, "y = " + to_string(a) + " * ln(x) + " + to_string(b)};
204 }
205
206 ApproxResult calc_power(const vector<Point>& pts) {
207     vector<Point> lin_pts;
208     for (const auto& p : pts) {
209         if (p.x <= 0 || p.y <= 0) return {"Степенная", {}, 0, 0, 0, false, "x, y <= 0
        ↪ "};
210         lin_pts.push_back({log(p.x), log(p.y)});
211     }
212     ApproxResult lin = calc_linear(lin_pts);
213     if (!lin.valid) return {"Степенная", {}, 0, 0, 0, false, ""};
214     double a = exp(lin.coeffs[1]), b = lin.coeffs[0], S = 0;
215     vector<double> phi_vals(pts.size());
216     for (size_t i = 0; i < pts.size(); i++) {
217         phi_vals[i] = a * pow(pts[i].x, b);
218         S += pow(phi_vals[i] - pts[i].y, 2);
219     }
220     return {"Степенная", {a, b}, S, sqrt(S / pts.size()), calc_R2(pts, phi_vals), true,
        ↪ "y = " + to_string(a) + " * x^(" + to_string(b) + ")"};
221 }
222
223 double evaluate_approx(const ApproxResult& res, double x) {
224     if (res.name == "Линейная") return res.coeffs[0] * x + res.coeffs[1];
225     if (res.name == "Квадратичная") return res.coeffs[0] * x * x + res.coeffs[1] * x +
        ↪ res.coeffs[2];
226     if (res.name == "Кубическая") return res.coeffs[0] * pow(x, 3) + res.coeffs[1] *
        ↪ pow(x, 2) + res.coeffs[2] * x + res.coeffs[3];
227     if (res.name == "Экспоненциальная") return res.coeffs[0] * exp(res.coeffs[1] * x);
228     if (res.name == "Логарифмическая") return res.coeffs[0] * log(x) + res.coeffs[1];
229     if (res.name == "Степенная") return res.coeffs[0] * pow(x, res.coeffs[1]);
230     return 0;
231 }
232
233 void plot_best_approximation(const vector<Point>& pts, const ApproxResult& best) {
234
235     vector<double> x_pts, y_pts;
236     double min_x = pts[0].x, max_x = pts[0].x;
237
238     for (const auto& p : pts) {
239         x_pts.push_back(p.x);
240         y_pts.push_back(p.y);
241         if (p.x < min_x) min_x = p.x;
242         if (p.x > max_x) max_x = p.x;
243     }
244
245     int N = 100;
246     double step = (max_x - min_x) / (N - 1);
247     vector<double> curve_x, curve_y;
248     for (int i = 0; i < N; i++) {
249         double x = min_x + i * step;
250         if (x <= 0 && (best.name == "Логарифмическая" || best.name == "Степенная")) x =
        ↪ 1e-6;
251         curve_x.push_back(x);
252         curve_y.push_back(evaluate_approx(best, x));
253     }
254
255     plt::figure();
256

```



```

257 plt::scatter(x_pts, y_pts, 30.0, {{"color", "red"}, {"label", "Data points"}});
258 plt::plot(curve_x, curve_y, {{"color", "blue"}, {"label", "Best approximation"}});
259
260 plt::xlabel("X");
261 plt::ylabel("Y");
262 plt::title("МНК");
263 plt::legend();
264 plt::grid(true);
265
266 plt::show();
267 }
268
269
270 int main() {
271     vector<Point> points;
272
273     int input_type;
274     while(true) {
275         cout << "Ввод данных:\n1 - С клавиатуры\n2 - Из файла\nВвод: ";
276         input_type = get_input<int>("");
277         if(input_type == 1 || input_type == 2) break;
278         cout << "Введите 1 или 2\n";
279     }
280
281     if (input_type == 1) {
282         int n;
283         while(true) {
284             n = get_input<int>("Введите количество точек (от 8 до 12): ");
285             if (n >= 8 && n <= 12) break;
286             cout << "Количество точек от 8 до 12\n";
287         }
288         for (int i = 0; i < n; i++) {
289             cout << "Точка " << i + 1 << ":\n";
290             double x = get_input<double>(" x = ");
291             double y = get_input<double>(" y = ");
292             points.push_back({x, y});
293         }
294     } else {
295         string filename;
296         cout << "Введите имя файла: ";
297         cin >> filename;
298         ifstream fin(filename);
299         if (!fin.is_open()) {
300             cout << "Ошибка открытия файла\n";
301             return 1;
302         }
303         double x, y;
304         while (fin >> x >> y) points.push_back({x, y});
305         fin.close();
306         if (points.size() < 8) {
307             cout << "Недостаточно точек в файле\n";
308             return 1;
309         }
310         cout << "Считано " << points.size() << " точек\n";
311     }
312
313     vector<ApproxResult> results;
314     results.push_back(calc_linear(points));
315     results.push_back(calc_quad(points));
316     results.push_back(calc_cubic(points));
317     results.push_back(calc_exp(points));

```

```

318 results.push_back(calc_log(points));
319 results.push_back(calc_power(points));
320
321 cout << "\n" << string(95, '-') << "\n";
322 cout << left << setw(18) << "Вид функции"
323     << setw(15) << "Меpa S"
324     << setw(15) << "Откл. delta"
325     << setw(12) << "R^2"
326     << "Уравнение" << "\n";
327 cout << string(95, '-') << "\n";
328
329 ApproxResult* best = nullptr;
330
331 for (auto& res : results) {
332     if (!res.valid) {
333         cout << left << setw(18) << res.name << res.equation << "\n";
334         continue;
335     }
336     cout << left << setw(18) << res.name
337         << setw(15) << fixed << setprecision(5) << res.S
338         << setw(15) << res.delta
339         << setw(12) << res.R2
340         << res.equation << "\n";
341
342     if (best == nullptr || res.delta < best->delta) best = &res;
343 }
344 cout << string(95, '-') << "\n";
345
346 double r = pearson_correlation(points);
347 cout << "\nКоэффициент Пирсона r = " << r << "\n";
348
349 if (best) {
350     cout << "\nНаилучшая аппроксимация: " << best->name << "\n";
351     cout << "Уравнение: " << best->equation << "\n";
352
353     plot_best_approximation(points, *best);
354 }
355
356 return 0;
357 }

```

3. Выводы

Во время выполнения работы мне удалось изучить различные виды аппроксимации: линейную, квадратичную, кубическую, логарифмическую, экспоненциальную и степенную. Нельзя однозначно сказать, какая аппроксимирующая функция лучше, так как это зависит от самих входных данных.